

WCount (C++, Windows)

A simple, consistent output display, dialog utility

Display and format text, numbers, and user-defined types to message boxes,
Output to clipboard, memos, or strings using one readable << >> syntax .

```
WCount << "Pi = " << 3.14159 << PI << SHOW;  
WCount << LongMessage >> CLIPBOARD;
```

Quick Start

- Add `WCount.cpp` to your project.
- Include `WCount.h` in any file where you want to use it.
- That's it—no external libs or complex setup required.

Every number, text using <<

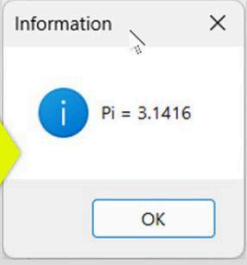
Every format, control using <<

Every output to Clipboard, Memo >>

Table of Contents

Message Display & Output (SHOW, Clipboard)	2
Message Formatting Text and Numbers	3
Simple Confirm Dialog	4
Custom Confirm Dialog	5
User Defined Type Display	6
Spaces in large blocks of text (touching text/data)	7

Message Display & Output (SHOW, Clipboard)

Category	Function	Usage
String	Width	<pre>WCout << W-30</pre> <i>W-30 stays active until << W-OFF</i>
Float	Format	<pre>double PI = 3.1416 WCout << FF-7.2 << PI << SHOW;</pre> <i>FF 7.2 sets width=7 decimals=2 (0..9) until FF-OFF</i>
Message	Display	<pre>WCout << "Pi = " << 3.1416; WCout << SHOW; WCout << SHOW-Warning; WCout << SHOW-Error; WCout << SHOW-Info; WCout << SHOW-Confirm; WCout << SHOW-Question;</pre>  
Text	Retrieve	<pre>Memo->Lines->Add(WCout.get()); Memo->Lines->Add(WCout()); UnicodeString Text=WCout.get(); WCout >> Text;</pre>
	Clipboard	<pre>WCout >> CLIPBOARD;</pre>
Message	Confirm	<pre>bool isConfirm = WCout.DLGConfirm(L"Do you want ketchup with the fries");</pre> <i>returns true or false</i>
WCout	Status	<pre>WCout << STATUS-ON; WCout << STATUS-OFF; WCout << STATUS-RESET;</pre> <i>Status affects all output globally (handy debugging)</i> <pre>STATUS-ON (Messages displayed) STATUS-OFF (Nothing displayed) STATUS-RESET (All settings default)</pre>

Message Formatting Text and Numbers

Message

Format data

C++

```
void TForm2::SHOWMessage()
{
    double PI = 3.14159265;

    WCount << CLEAR;
    WCount << FF-10.4 << PI << EL //Float Format 10 wide 4 decimals
    << FF-4.3 << PI << EL
    << FF-12.1 << PI << EL
    << FF-OFF << PI << EL
    << SHOW;

    WCount << L"The following items are in your shopping cart" << EL

    << FF-2.2
    << L"=== RPAD-3 (3 Trailing spaces) ===" << EL
    << RPAD-3 //3 Trailing spaces after text;
    << L" Cauliflower " << 5.6987 << EL
    << L" BlueBerries " << 12.325 << EL
    << L" Greek Yogurt " << 12. << EL
    << L" Vega Protein Powder " << 65.92 << EL << EL
    << RPAD-OFF //Turn Right Pad OFF
    << SHOW

    WCount << L"=== FF-NO turn off Floating point formatting ===" << EL
    << FF-OFF //Format Float Spaces OFF
    << RPAD-20 << L" PAD Cauliflower " << TAB << 5.6987 << EL
    << RPAD-OFF
    << L" BlueBerries " << TAB << 12.325 << EL
    << L" Greek Yogurt " << TAB << 12. << EL
    << L" Vega Protein Powder " << 65.92 << EL << EL
    << SHOW

    WCount << L"=== W-30 (String Width 30) FF-7.2 (Float 7 wide 2 dec.)
    ===" << EL
    << FF-7.2 //Floating Point Format 7 wide 2 decimals
    << L" W-30 impact more visible in FIXED PITCH FONT " << NL
    << W-30 //String Width = 30
    << L" ITEM " << L"THE Amount" << NL
    << L"===== " << L"===== " << NL
    << L"Cauliflower" << 5.6987 << NL
    << L" BlueBerries" << 12.325 << NL
    << L" Greek Yogurt" << 12. << NL
    << L" Vega Protein Powder" << 65.92 << EL
    >> CLIPBOARD;
}
```

WCount << SHOW;
WCount >> CLIPBOARD;

Testwcount
3.1416
3.142
3.1
3.14159265
0
OK

Testwcount
The following items are in your shopping cart
=== RPAD-3 (3 Trailing spaces) ===
Cauliflower 5.70
BlueBerries 12.32
Greek Yogurt 12.00
Vega Protein Powder 65.92
OK

Testwcount
=== FF-NO turn off Floating point formatting ===
PAD Cauliflower 5.6987
BlueBerries 12.325
Greek Yogurt 12
Vega Protein Powder 65.92
OK

NOTEPAD | NOTEPAD++ | Courier Font
These numbers would line up perfectly with a
Fixed-Pitch font such as Courier

ITEM
File Edit View
ITEM Amount
=====

Cauliflower	5.70
BlueBerries	12.32
Greek Yogurt	12.00
Vega Protein Powder	65.92

Ln 3, Col 32 262 cha Plair 100% Win UTF-8

Simple Confirm Dialog

Category	Function	Usage
Message	Confirm (Simple)	<pre>bool isConfirm = Wcout.DLGConfirm(L"Do you want ketchup with the fries");</pre>

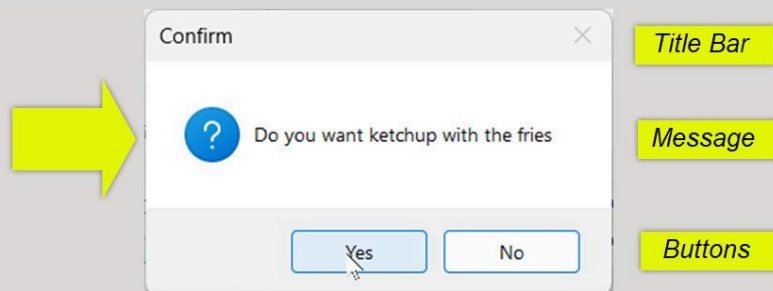
2 optional Parameters

```
bool isConfirm = Wcout.DLGConfirm(L"Do you want ketchup with the fries");
bool isConfirm = Wcout.DLGConfirm(L"Message"); [optional]
bool isConfirm = Wcout.DLGConfirm(L"Message", [L"TitleBar"]);
bool isConfirm = Wcout.DLGConfirm(L"Message", [btnYES_NO]); [optional]
bool isConfirm = Wcout.DLGConfirm(L"Message", [L"TitleBar"], [btnYES_NO]);
```

DEFAULT TitleBar="Confirm"
DEFAULT buttons=OK/Cancel

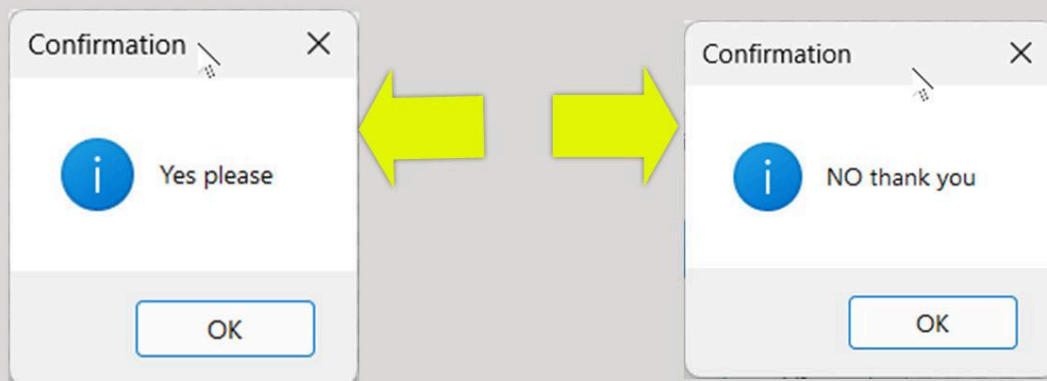
```
void __fastcall TForm2::btnTESTClick(TObject *Sender)
{
    bool isConfirm = Wcout.DLGConfirm(L"Do you want ketchup with the fries", btnYES_NO);
    .....
}
```

C++



```
.....
Txt Msg = isConfirm ? L"Yes please" : L"NO thank you";
Wcout << Msg << SHOW-Confirm;
```

C++



Custom Confirm Dialog

Category	Function	Usage
Type	USER Defined	
<pre> struct Point2D { int x, y; }; TWCout& operator<<(TWCout& wc, const Point2D& p) { wc << L"x=" << p.x << L", " << L"y=" << p.y << L"; return wc; } void __fastcall TForm2::btnTESTMeClick(TObject *Sender) { Point2D p1 {3, 5}; Point2D p2 {10, 20}; WCount << L"The Coordinates of the line are: " << FF-7.2 << EL << L"Start " << p1 << EL << L"End " << p2 << SHOW-Info; } </pre> C++ <i>The Magic of operator<< Overloading</i> <i>You Write this function once - it can be a bit tricky the first time.</i> <i>After that?</i> <i>Your type works everywhere with << - chaining, formatting (FF-, W-), SHOW, clipboard, memos - zero extra effort.</i> <pre> class Complex { public: Complex(int Real, int Imaginary) { this->Real = Real; this->Imaginary = Imaginary; } int Real; int Imaginary; }; TWCout& operator<<(TWCout& wc, Complex C) { wc << C.Real << L" + " << C.Imaginary << L"i"; return wc; } void TForm2::ShowComplex() { Complex C {3, 4}; WCount << L"Complex = " << C << SHOW; } </pre> C++ <i>Any user type (TWCout&<<UserType) defined</i> <i>Simplified display</i> <i>The overload is written once - then your type becomes as easy to use as int or String forever</i> <i>Simplified display</i> C++ <i>Simplified display</i>		
<pre> class Complex { public: Complex(int Real, int Imaginary) { this->Real = Real; this->Imaginary = Imaginary; } int Real; int Imaginary; }; TWCout& operator<<(TWCout& wc, Complex C) { wc << C.Real << L" + " << C.Imaginary << L"i"; return wc; } void TForm2::ShowComplex() { Complex C {3, 4}; WCount << L"Complex = " << C << SHOW; } </pre> C++ <i>Any user type (TWCout&<<UserType) defined</i> <i>Simplified display</i> <i>Simplified display</i> C++ <i>Simplified display</i>		

Simplified display

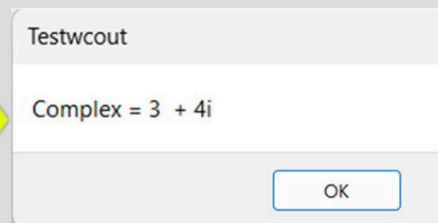
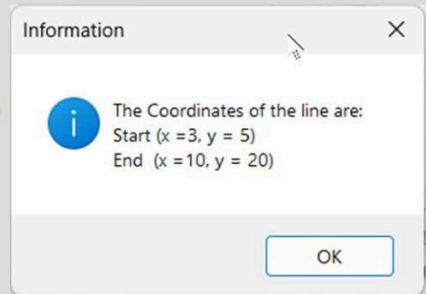
C++

Simplified display

Simplified display

C++

Simplified display



User Defined Type Display

Category	Function	Usage
Type	USER Defined	

C++

```

struct Point2D {
    int x, y;
};

TWOut& operator<<(TWOut& wc, const Point2D& p) {
    wc << L"x=" << p.x << L", " << L"y=" << p.y << L"";
    return wc;
}

void __fastcall TForm2::btnTESTMeClick(TObject *Sender)
{
    Point2D p1 {3, 5};
    Point2D p2 {10, 20};

    WOut << L"The Coordinates of the line are: " << FF-7.2 << EL
        << L"Start " << p1 << EL
        << L"End " << p2 << SHOW-Info;
}

```

The overload is written once - then your type becomes as easy to use as int or String forever

The Magic of operator<< Overloading
 You Write this function **once** - it can be a bit tricky the first time.
 After that?
 Your type works **everywhere** with << - chaining, formatting (FF-, W-), SHOW, clipboard, memos - **zero extra effort**.

Simplified display

Information

The Coordinates of the line are:
 Start (x =3, y = 5)
 End (x =10, y = 20)

OK

C++

```

class Complex {
public:
    Complex(int Real, int Imaginary) {
        this->Real = Real;
        this->Imaginary = Imaginary;
    }
    int Real;
    int Imaginary;
};

TWOut& operator<<(TWOut& wc, Complex C)
{
    wc << C.Real << L" + " << C.Imaginary << L"i";
    return wc;
}

void TForm2::ShowComplex() {
    Complex C {3, 4};
    WOut << L"Complex = " << C << SHOW;
}

```

Any user type (TWOut&<<UserType) defined

Simplified display

Testwcout

Complex = 3 + 4i

OK

Spaces in large blocks of text (touching text/data)

Text

SPACING

WCout << AUTOSPACE-ON;
Spacing will automatically be inserted between touching text\data

WCout

Use WCout Text Modes, Formats Show Message types

TWCout
☒ ON
☐ OFF

WCout << AUTOSPACE-OFF;
☒ OFF
☐ ON

☐ Set Width
30

Floating point
☐ Format
Width Decimals
7 2

Letter Formatting

Hello Michel Welcome to our forum Mr. Pinard.
It is a pleasure to have you join us Michel to discuss vintage stereo chat group

Note that the price of a90day subscription to manufacturer discounts will increase from6to about7.25per month
IN the 70s equipment designed by engineers and not bean counters we agree Michel Pinard that these were vastly superior to today's hardware.

WCout << L"Note that the price of a" << 90 << "day subscription to manufacturer discounts will increase from" << 6.00 << "to about" << 7.25 << "per month" << "IN the 70s equipment designed by engineers and not bean counters" << "we agree Michel Pinard that these were vastly superior to today's hardware." << EL << EL;
C++
Spacing was not adjusted

Once again thank you for joining our forum Michel

WCout

Use WCout Text Modes, Formats Show Message types

TWCout
☒ ON
☐ OFF

AUTO Insert space
☐ OFF
☒ ON
WCout << AUTOSPACE-ON;

☐ Set Width
30

Floating point
☐ Format
Width Decimals
7 2

Letter Formatting

Hello Michel Welcome to our forum Mr. Pinard.
It is a pleasure to have you join us Michel to discuss vintage stereo chat group

Note that the price of a 90 day subscription to manufacturer discounts will increase from 6 to about 7.25 per month
IN the 70s equipment designed by engineers and not bean counters we agree Michel Pinard that these were vastly superior to today's hardware.

WCout << L"Note that the price of a" << 90 << "day subscription to manufacturer discounts will increase from" << 6.00 << "to about" << 7.25 << "per month" << "IN the 70s equipment designed by engineers and not bean counters" << "we agree Michel Pinard that these were vastly superior to today's hardware." << EL << EL;
C++
Spacing was automatically inserted between touching text\data